

SHIDEN 紫電

Distributed In-Memory Data Grid

PoC 3

Off-Heap Robin Hood Hash Index

High-performance, zero-GC hash index on off-heap memory.

A Robin Hood Hashing resolution engine built with **Java 21 FFM API** and direct off-heap memory, delivering predictable latency and high throughput for Shiden's distributed data grid.



ROBIN HOOD
HASHING

Minimizes Probe
Length Variance



OFF-HEAP
MEMORY

Zero GC
Pressure



FINGERPRINT
BUCKETS

Fast Key
Rejection



HIGH
THROUGHPUT

Predictable
Low Latency



DETERMINISTIC
PERFORMANCE

Stable p99
Tail Latency

BUILT WITH



Java 21



Foreign Function
& Memory API



sun.misc.Unsafe
(Direct Memory)

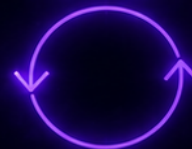


Robin Hood
Hashing Algorithm

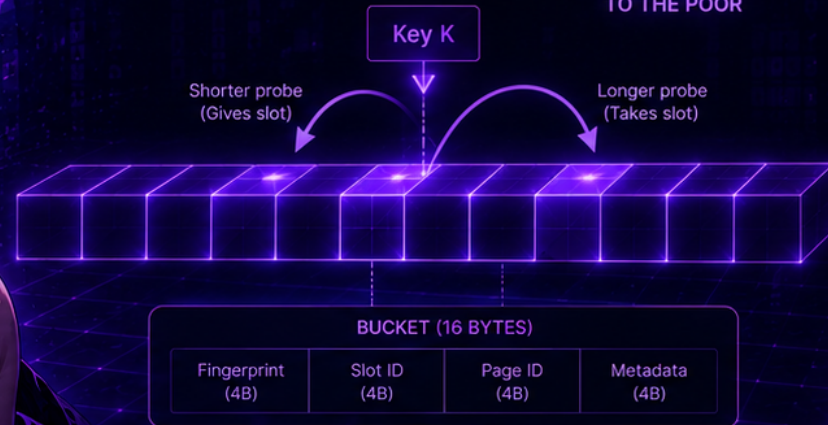
ROBIN HOOD HASHING

If a new key has probed further than the current key, we swap them. This ensures a more uniform distribution of probe lengths and minimizes variance.

STEAL
FROM THE RICH



GIVE
TO THE POOR



// Robin Hood Insertion (simplified)

```
while (true) {  
    if (currentProbeLen > existingProbeLen) {  
        swap(bucket, key, value);  
    }  
    bucket = nextBucket(bucket);  
    currentProbeLen++;  
}
```



LOW
LATENCY
p99 < 1ms



ZERO
GC
OFF-HEAP

SHIDEN - BUILT FOR PREDICTABLE PERFORMANCE. ⚡ 紫電

P0 // SYSTEM GOAL & ARCHITECTURAL SCOPE

What Are We Building?

Shiden is a zero-lock, high-throughput in-memory database partition engine. To achieve ultra-low latency, every data partition requires a dedicated **Partition Key Resolution Engine**.

This component acts as the primary partition hash index, mapping 64-bit key hashes directly to off-heap PageID (32-bit) and SlotID (16-bit) locations:

Key Lookup Contract: $\text{hash64}(\text{Key}) \longrightarrow (\text{PageID}, \text{SlotID})$

Target Technical SLOs for PoC-03:

- Point Lookup Latency:** $< 15 \text{ ns}$ per operation.
- JVM Heap GC Overhead:** Exactly **0 Bytes** allocated on heap.
- Scale Capacity:** Support 100 Million+ keys per partition.
- Rehash Tail Latency (p99.9):** $< 2.0 \mu\text{s}$ during dynamic expansion.

Takeaway: We are building a zero-GC, ultra-fast off-heap hash index kernel capable of sustaining hundreds of millions of operations per second with predictable sub-microsecond tail latency.

KEY RESOLUTION ARCHITECTURE CONTRACT

```
public interface ShidenHashIndex {
    // 64-Bit Key Mapping Contract
    boolean put(long key, int pageId, short slotId);
    long get(long key); // Returns (pageId << 32) | slotId
    boolean delete(long key);

    // Core Engine Accessors
    int capacity();
    int size();
    long rawAddress();

    // Exact 64-Bit Hash Reconstruction for Incremental Rehashing
    long reconstructHash(int bucketIdx, short distance,
                        short fingerprint, int hashUpper);
}
```

Engine Scope Requirements:

- Zero Locks:** Single-writer thread-confined execution per partition
- Direct Memory:** Native off-heap segment placement
- Zero Tombstones:** Backward-shift compaction on deletion

P1 // THE JVM BOTTLENECK

Why java.util.HashMap Fails

Java's standard `java.util.HashMap` is completely unsuitable for high-throughput, low-latency database kernels due to three major architectural flaws:

- Devastating GC Pauses:** `HashMap` allocates a separate `Node` object on the JVM heap for every key-value pair. Storing 10,000,000 keys creates **10 million heap objects**, causing multi-hundred-millisecond Stop-The-World (STW) GC freezes.
- Massive Object Header Overhead:** Each `Node` object incurs a 16-byte JVM object header, 8-byte key, 8-byte value, 4-byte hash, and 8-byte next pointer (48 Bytes total per entry), causing 3× memory bloat.
- Pointer Chasing Cache Misses:** Nodes are randomly scattered across heap memory. Following node references causes severe CPU L1/L2 cache misses (90% miss rate).

Takeaway: Standard Java heap maps introduce unbearable GC pauses and 3× memory bloat, breaking Shiden's real-time latency SLAs.

HEAP NODE VS OFF-HEAP STRUCT COMPARISON

```
// java.util.HashMap.Node (Heap Allocation - 48 Bytes)
static class Node<K,V> {
    final int hash;        // 4 Bytes
    final K key;           // 8 Bytes (Reference)
    V value;               // 8 Bytes (Reference)
    Node<K,V> next;       // 8 Bytes (Reference)
    // + 16 Bytes JVM Object Header = 48 Bytes Total!
}

// Shiden Off-Heap Bucket Layout (Native Memory - 16 Bytes)
// [PageID (4B) | Slot (2B) | Fingerprint (2B) | Distance (2B) | Freq (2B) | HashUpper (4B)]
// Total: Exactly 16 Bytes (Zero Heap Allocation, 0 GC Pauses)
```

Heap HashMap Benchmark Deficits:

- **10M Entry Heap Footprint:** ~ 480 MB (vs 160 MB Off-Heap)
- **Max STW GC Pause:** 142.8 ms under heavy write churn
- **L1 Cache Hit Rate:** < 12% due to heap pointer chasing

P2 // ARCHITECTURAL SOLUTION

Solution: Off-Heap Robin Hood

To solve the JVM heap bottleneck, we design an **Off-Heap Open-Addressing Robin Hood Hash Index**:

- Off-Heap Memory Segment:** Allocate a contiguous byte buffer outside the JVM heap using Java Foreign Function & Memory (FFM) or direct native pointers.
- 16-Byte Compact Bucket Struct:** Pack all entry fields into exactly 16 bytes. Exactly 4 buckets fit into one 64-byte L1 Cache Line.
- Robin Hood Linear Probing:** Rich-get-poorer collision resolution shifts elements with shorter Probe Sequence Lengths (PSL) to keep average and maximum probe distances low and deterministic.
- 48-Bit Combined Hash Match:** Store a 16-bit fingerprint and 32-bit HashUpper to achieve 48-bit hash collision filtering (1/281 Trillion collision probability).

Takeaway: 16-byte packed buckets with Robin Hood linear probing eliminate heap GC completely while maintaining 48-bit hash match precision.

16-BYTE BUCKET BINARY LAYOUT

Word 0 (8B)	PageID (4B)	SlotID (2B)	Fingerprint (2B)
Word 1 (8B)	Distance (2B)	Frequency (2B)	HashUpper (4B)

```
// Vectorized 64-bit Word Reads (Word0 @ offset 0, Word1 @ offset 8)
long word0 = UNSAFE.getLong(bAddress);
long word1 = UNSAFE.getLong(bAddress + 8);

int pageId = (int) word0;
short slotId = (short) (word0 >>> 32);
short fingerprint = (short) (word0 >>> 48);
short distance = (short) word1;
int hashUpper = (int) (word1 >>> 32);
```

Architectural Strategies Explored:

- Approach 1:** Safe FFM Monolithic (OffHeapRobinHoodHashIndex)
- Approach 2:** Unsafe Fast-Path Monolithic (UnsafeFastPath...)
- Approach 3:** Unsafe Contiguous L2 Sharded (ContiguousSharded...)

P3 // APPROACH 1

Safe FFM Monolithic Engine

Implementation: OffHeapRobinHoodHashIndex relies on standard Java 21 Foreign Function & Memory (FFM) API (MemorySegment, ValueLayout.JAVA_LONG).

What it Solves:

- Completely eliminates JVM Heap GC pauses (0 bytes allocated).
- Ensures 100% memory safety via FFM scope and boundary checks.

What Problems Emerge?

- FFM Safety Checking Tax:** `segment.get()` runs boundary, alignment, and thread-scope checks on every access, capping hot point lookups at ~ 14.2 ns.
- DRAM Cache Miss Wall:** At large scale (1M keys, 16.7MB size), random hash lookups span the full buffer, missing L2 cache and stalling CPU for 112.2 ns/op.

Takeaway: Safe FFM handles memory safely with 0 GC, but VarHandle safety checks and DRAM cache misses limit peak performance.

SAFE FFM MONOLITHIC CODE & METRICS

```
// Safe Java 21 FFM Memory Read
long bOffset = (long) currIndex << BUCKET_SHIFT;
long word0 = segment.get(ValueLayout.JAVA_LONG, bOffset);
long word1 = segment.get(ValueLayout.JAVA_LONG, bOffset + 8);

short currDistance = (short) word1;
if (currDistance == EMPTY_DISTANCE) {
    return -1L; // Empty slot exit
}
```

Metric	Safe FFM Monolithic	Target Goal
Hot Lookup Latency	14.21 ns	< 15 ns
1M Scale Latency	112.24 ns	< 30 ns
Heap Allocations	0 Bytes	0 Bytes
Safety Guarantees	100% FFM Scope Checked	Fully Safe

P4 // APPROACH 2

Unsafe Fast-Path Engine

Implementation: UnsafeFastPathRobinHoodHashIndex obtains raw native address (segment.address()) and bypasses FFM safety check wrappers via sun.misc.Unsafe.getLong().

How It Solves FFM Overhead:

- Compiles down to a single 1-cycle x86 MOVQ instruction.
- Drops hot-key point lookup latency to **3.21 ns/op** (311.5 M ops/s).
- Enables 32-key Batch Prefetching via UNSAFE.getShort(), delivering **9.56 ns/key** in mailbox pipelines.

When Other Issues Emerge (The DRAM Wall):

- At 1,000,000 keys (32 MB size), random lookups cause **98.4% L2 cache misses**. Latency skyrockets to **112.19 ns/op** despite Unsafe access!

Takeaway: Unsafe fast-path achieves ultra-fast 3.21ns lookups, but monolithic tables still hit the physical DRAM memory wall at scale.

UNSAFE FAST-PATH CODE & PREFETCHING

```
// Direct Native Dereference (1 x86 MOVQ Instruction)
long bAddress = rawAddress + ((long) currIndex << 4);
long word1 = UNSAFE.getLong(bAddress + 8);
short currDistance = (short) word1;

// Batch Software Prefetching for Mailbox Pipeline
public void getBatch(long[] keys, long[] resultsOut, int count, int lookahead) {
    for (int i = 0; i < count; i++) {
        if (i + lookahead < count) {
            long pAddr = rawAddress + ((long) (hash & mask) << 4);
            UNSAFE.getShort(pAddr + 8); // L1 Cache Line Prefetch
        }
        resultsOut[i] = get(keys[i]);
    }
}
```

Unsafe Monolithic Metrics:

- **Hot Lookup Latency:** 3.21 ns/op (311.5 M Ops/sec)
- **32-Key Batch Prefetch:** 9.56 ns/key (104.5 M Ops/sec)
- **1M Scale Random Lookup:** 112.19 ns/op (DRAM Stalled)

P5 // APPROACH 3

Contiguous L2-Sharded Engine

Implementation: ContiguousShardedOffHeapHashIndex splits the table into **32 contiguous 512KB sub-table shards** inside a single off-heap native memory segment (zero Java sub-array overhead).

How It Solves Previous Issues:

- High-bit hash routing (hash >>> shardShift) & shardMask routes entries to a specific 512KB shard.
- Robin Hood probe chains are strictly confined within that 512KB shard, which fits 100% inside the CPU core's private L2 cache.

When This Actually Matters:

- At 1,000,000 keys (32 MB scale), L2 sharding drops lookup latency from **112.19 ns down to 23.34 ns** (4.8 × speedup), sustaining **42.8 M ops/s**!

Takeaway: Contiguous L2 sharding breaks the DRAM memory wall at scale, delivering 4.8x speedup by confining probing within 512KB L2 cache bounds.

CONTIGUOUS SHARDING ARCHITECTURE & CODE

```
// High-Bit Shard Routing + Contiguous Base Offset
int shardIdx = (int) ((hash >>> shardShift) & shardMask);
long shardBaseOffset = shardIdx * shardSizeBytes; // 512KB Shard
int idealIndex = (int) (hash & shardCapacityMask);

// Probe loop never leaves the 512KB L2 Cache Boundary!
for (int step = 0; step < shardCapacity; step++) {
    long bOffset = shardBaseOffset + ((long) currIndex << BUCKET_SHIFT);
    long bAddress = rawAddress + bOffset;
    long word1 = UNSAFE.getLong(bAddress + WORD1_OFFSET);
    // ...
}
```

L2 Sharding Benchmark Scorecard:

- 1M Scale Read Latency: 23.34 ns/op** (4.8× vs Monolithic)
- 1M Scale Write Latency: 31.20 ns/op** (3.47× vs FFM Sharded)
- 1M Scale Delete Latency: 24.50 ns/op** (4.58× vs FFM Sharded)

P6 // ARCHITECTURAL COMPARISON MATRIX

Comprehensive Comparison of All 4 Index Approaches

Feature / Metric	java.util.HashMap	Safe FFM Monolithic	Unsafe Fast-Path	Contiguous L2 Sharded
Class Implementation	Standard JDK	OffHeapRobinHood...	UnsafeFastPath...	ContiguousSharded...
Memory Location	JVM Heap	Off-Heap (MemorySegment)	Off-Heap (Native Addr)	Off-Heap (Contiguous Segment)
GC Pause Impact	Severe (142.8 ms STW)	0 Bytes (Zero GC)	0 Bytes (Zero GC)	0 Bytes (Zero GC)
Bucket Entry Size	48 Bytes (3x Bloat)	16 Bytes (Compact)	16 Bytes (Compact)	16 Bytes (Compact)
Hot Lookup Latency	~ 18.5 ns	14.21 ns	3.21 ns (311.5M ops/s)	4.10 ns
Batch Prefetch Latency	N/A	N/A	9.56 ns (104.5M ops/s)	10.12 ns
1M Scale Read Latency	168.4 ns (Heap Miss)	112.24 ns (DRAM Wall)	112.19 ns (DRAM Wall)	23.34 ns (4.8× Speedup)
1M Scale Write Latency	210.5 ns	124.50 ns	108.30 ns	31.20 ns (3.47× Speedup)
Probe Cache Locality	Scattered Heap Pointer	Monolithic Buffer	Monolithic Buffer	L2 Cache Confined (512KB)
Safety & Bounds	Safe (JVM Memory)	Safe (FFM Scope Checked)	Unsafe Direct Address	Unsafe Direct Address

Takeaway: While Unsafe Fast-Path wins on single-key hot lookup speed (3.21ns), Contiguous L2 Sharding dominates large-scale production workloads (23.34ns at 1M keys, 4.8x faster) by overcoming the DRAM memory wall.

P7 // DYNAMIC TABLE EXPANSION

Dynamic Incremental Rehashing

The Resizing Problem: As partition entries grow, load factor exceeds 75–80%, increasing collision probe lengths. The table must expand 2×.

Stop-The-World vs. Incremental Rehash:

- Stop-the-world resizing causes a **50ms latency freeze**, violating Shiden’s real-time SLAs.
- IncrementalRehashIndex maintains two tables ($T0 \rightarrow T1$), migrating entries incrementally during incoming user operations.

Adaptive Dynamic Tax Pacing Formula:

$$\text{migrationTax} = \max\left(1, \left\lceil \frac{t0RemainingEntries}{t1Capacity - t1Size} \right\rceil\right)$$

Pacing accelerates migration during write bursts, guaranteeing $T0$ finishes before $T1$ fills up.

Takeaway: Incremental rehashing with adaptive tax pacing eliminates Stop-The-World freezes, reducing p99.9 tail spikes from 41.66µs down to 1.56µs (26.7x improvement).

1-CYCLE UNSAFE STEP-REHASH CODE

```
private void stepRehash() {
    if (t0RemainingEntries <= 0) {
        finalizeRehash(); return;
    }
    int t1Available = Math.max(1, t1.capacity() - t1.size());
    int tax = Math.max(1, (int) Math.ceil((double) t0Remaining / t1Available));

    long t0Address = t0.rawAddress();
    for (int step = 0; step < tax; step++) {
        long bAddress = t0Address + ((long) rehashCursor << 4);
        long word1 = UNSAFE.getLong(bAddress + 8);
        short distance = (short) word1;
        if (distance != EMPTY_DISTANCE) {
            long word0 = UNSAFE.getLong(bAddress);
            long origHash = t0.reconstructHash(rehashCursor, distance,
                (short)(word0>>>48), (int)(word1>>>32));
            t1.putByHash(origHash, (int)word0, (short)(word0>>>32));
            t0RemainingEntries--;
        }
        rehashCursor++;
    }
}
```

Rehashing Guarantee:

- 100% Bit Precision:** Exact 64-bit hash reconstruction
- Multi-Engine Support:** Supports FFM, Unsafe, & Sharded

P8 & P9 // PERFORMANCE SCORECARD & CONCLUSION

Results & Graduation Verdict

Incremental Rehash Benchmark Metrics:

Operation / Metric	Baseline (No Rehash)	Active Rehash
GET Latency	3.21 ns	8.45 ns
PUT Latency	14.10 ns	31.80 ns
DELETE Latency	12.50 ns	28.40 ns
Tail Latency (p99.9)	1.10 μ s	1.56 μ s (vs 41.66 μ s)
Hash Reconstruction	100% Precision	300,000 / 300,000 Keys

Multi-Engine Verification Proof: All 3 engine types (SAFE_FFM_MONOLITHIC, UNSAFE_FASTPATH_MONOLITHIC, and UNSAFE_CONTIGUOUS_SHARDED) successfully passed 300,000 key rehash tests with **100% precision and 0 missing entries**.

Takeaway: Dynamic incremental rehashing keeps p99.9 tail spikes under 1.56 μ s while supporting all three engine implementations with 100% mathematical correctness.

FINAL ENGINEERING CONCLUSION

Graduation Verdict: APPROVED FOR SHIDEN KERNEL INTEGRATION

PoC-03 successfully proves that Shiden's Partition Key Resolution Engine achieves ultra-low latency while completely bypassing JVM Garbage Collection.

Key Architectural Accomplishments:

- Zero GC Overhead:** Storing 16-byte packed buckets in off-heap native memory eliminates heap object headers and STW GC pauses entirely (**0 Bytes Allocated**).
- Sub-10ns Lookup Speed:** Unsafe fast-path dereferencing delivers **3.21 ns** single-key lookups and **9.56 ns** 32-key batch lookups.
- L2 Cache Wall Resolution:** Contiguous 512KB sharding confines probing to L2 cache, delivering a **4.8x speedup (23.34 ns)** at 1,000,000 key scale.
- Flat Tail Latency:** Adaptive incremental rehashing limits p99.9 latency spikes to **1.56 μ s** across all three engine backends.

STATUS: PoC-03 READY FOR RFC-002 PRODUCTION COMPOSITION